

Optimization Problem - DVR - 16-6-26

Jason Joy Manoj

June 2026

1 Introduction

1.1 Accuracy

In chain routing, Section 3.1 describes a router (a fine-tuned BERT classifier) that takes the prompt and chain output so far, and outputs soft scores for each candidate model. Section 3.3, Equation (1) defines one of these soft scores, $\text{Sim}_i(x) \in [0, 1]$, representing the predicted similarity between a candidate model’s output and the gold-standard answer.

The accuracy term for DVR follows this same structure. For every stage $s \in \{\text{Exploration, Reasoning, Conclusion}\}$ and every candidate pair (v, k) , where v is the verifier and k is the draft chunk length, the router predicts a soft accuracy score:

$$A_{v,k}(s) \in [0, 1] \tag{1}$$

$A_{v,k}(s)$ is a number between 0 and 1 that the router predicts for each candidate (v, k) at stage s , given the supporting variables $[q(s-1), H(s-1), \alpha(s-1), s]$ as input. A higher value indicates a better expected accuracy if that choice is made.

In chain routing, Section 3.2 describes how training labels for $\text{Sim}_i(x)$ are gathered: prompts from several datasets are sent to each candidate LLM, and the similarity between the output and the gold-standard answer is recorded. The similarity measure used is BERTScore, defined in Section 4.7.1, Equation (6):

$$\text{BERTScore}(m, g) = \frac{\mathbf{e}_m \cdot \mathbf{e}_g}{\|\mathbf{e}_m\| \|\mathbf{e}_g\|} \tag{2}$$

where m is a model’s output, g is the gold-standard answer, and $\mathbf{e}_m, \mathbf{e}_g$ are their respective sentence embeddings.

For DVR, training labels for $A_{v,k}(s)$ are gathered the same way: the pipeline is run repeatedly on a dataset with known correct answers (e.g. GSM8K, also used by wheeler2025chain), trying different (v, k) combinations, and Equation 2 is used to score each run’s output against the gold-standard answer. These scores serve as the training labels. Once trained, at inference time the router

takes $[q(s-1), H(s-1), \alpha(s-1), s]$ as input and outputs $A_{v,k}(s)$ - its prediction of how good each candidate (v, k) would be, without needing to actually execute that candidate and measure it directly.

1.2 Compute Cost

Unlike accuracy, the compute cost term does not require a trained predictor. For each candidate (v, k) – a combination of verifier $v \in \{\text{none, small, medium, large}\}$ and draft chunk length $k \in \{3, 5, 7\}$ – the raw compute cost $C(v, k)$ (e.g. FLOPs required for verifier v to process a chunk of k tokens) is a known, fixed value that can be calculated in advance.

In chain routing, Section 3.4, Equation (4) defines a general-purpose soft score formula, stating explicitly that "the input score is the calculated similarity score for Sim_i or the environmental cost for Env_i " – i.e. the formula is designed to normalise any raw scalar input. For DVR, this formula is applied directly with $C(v, k)$ as the input score:

$$\text{Cost}_{v,k}(s) = \frac{\exp(C(v, k)/T)}{\sum_{(v', k')} \exp(C(v', k')/T)} \quad (3)$$

Equation 3 converts the raw compute cost $C(v, k)$ of each candidate into a value in $[0, 1]$, such that the values across all candidates (v', k') sum to 1. The temperature T controls the sharpness of this normalisation: a small T pushes the highest-cost candidate's value close to 1 and all others close to 0, while a large T pushes all values towards being equal. A higher $\text{Cost}_{v,k}(s)$ indicates that candidate (v, k) is *relatively* more expensive than the other candidates. In chain routing (Appendix B.1, where $T = 4.5$), T is a hyperparameter that may be tuned for DVR.

Note that $\text{Cost}_{v,k}(s)$ is a cost score, not a benefit score: a higher value is worse. This is consistent with how $\text{Env}_i(x)$ is treated in chain routing, where it is inverted (via $1 - \text{Env}_i(x)$) when combined into the final score in their Equation (2).

1.3 Energy

Like compute cost, the energy term does not require a trained predictor – it is a known, measurable quantity for each candidate (v, k) .

In chain routing, Section 4.8, Equation (8) defines how environmental (carbon) cost is calculated:

$$\text{CO}_2(\text{g}) = \frac{\text{power (kW)} \times \text{time (h)}}{1000} \times \text{Carbon Intensity (kg/kWh)} \quad (4)$$

Chain routing measures this directly using the CodeCarbon library, which "works by recording the power consumption of CPU, GPU, and RAM" (Section 4.8). For DVR, $E(v, k)$ is this $\text{CO}_2(\text{g})$ value for verifier v processing a chunk of

length k , measured the same way using CodeCarbon – a real, measured number, not an estimate.

As with Compute Cost, this raw value is normalised using the general-purpose soft score formula from Section 3.4, Equation (4):

$$\text{Energy}_{v,k}(s) = \frac{\exp(E(v,k)/T)}{\sum_{(v',k')} \exp(E(v',k')/T)} \quad (5)$$

Equation 5 converts the raw carbon cost $E(v,k)$ of each candidate into a value in $[0, 1]$, such that the values across all candidates sum to 1, using the same temperature parameter T described previously. A higher $\text{Energy}_{v,k}(s)$ indicates that candidate (v,k) produces relatively more carbon emissions than the other candidates.

Of the three resource terms (Compute Cost, Energy, Time), Energy has the strongest direct grounding in chain routing: $E(v,k)$, via Equation 4 and CodeCarbon, is exactly $\text{Env}_i(x)$ as used in chain routing’s own results – the same quantity their reported $\lambda = 0.7$ trade-off (Section 5.2) is tuned against. As with $\text{Cost}_{v,k}(s)$, a higher $\text{Energy}_{v,k}(s)$ is worse, and will be inverted (via $1 - \text{Energy}_{v,k}(s)$) when combined into the final score, consistent with how chain routing treats $\text{Env}_i(x)$ in their Equation (2).

1.4 Time

Unlike Compute Cost and Energy, the Time term is not derived from chain routing’s normalisation formula. Instead, it follows directly from the foundational speculative decoding paper, [leviathan2023](#), which derives a closed-form expression for the expected speedup of speculative decoding over standard autoregressive decoding.

[leviathan2023](#) show that, given an acceptance rate α , a draft length γ , and a cost ratio c (the time for one draft-model pass divided by the time for one verifier-model pass), the expected speedup is:

$$\text{Speedup}(\alpha, \gamma, c) = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)} \quad (6)$$

For DVR, this formula is applied directly with $\gamma = k$, $\alpha = \alpha(s - 1)$, and $c = L(\text{draft})/L(v)$, giving:

$$\text{Time}_{v,k}(s) = \frac{1 - \alpha(s - 1)^{k+1}}{(1 - \alpha(s - 1)) \left(k \cdot \frac{L(\text{draft})}{L(v)} + 1 \right)} \quad (7)$$

Equation 7 gives the expected speedup factor for choosing verifier v with chunk length k at stage s , based on the acceptance rate observed in the previous stage. A higher acceptance rate $\alpha(s - 1)$ increases the expected speedup, since more drafted tokens survive verification per call. A larger cost ratio $L(\text{draft})/L(v)$ (i.e. verifier v is slow relative to the drafter) reduces the speedup, since the $\left(k \cdot \frac{L(\text{draft})}{L(v)} + 1 \right)$ term in the denominator grows.

Note: Unlike $\text{Cost}_{v,k}(s)$ and $\text{Energy}_{v,k}(s)$, which are normalised to $[0, 1]$ with higher values representing worse (more costly) candidates, $\text{Time}_{v,k}(s)$ is typically greater than 1 when speculative decoding provides a speedup, and can fall below 1 if it does not – with higher values representing *better* (faster) candidates. This difference in orientation and range must be addressed when combining $\text{Time}_{v,k}(s)$ with the other terms in the final combined score.

2 Combined Optimization Problem

Following chain routing’s structure exactly, the three resource/efficiency terms – $\text{Cost}_{v,k}(s)$, $\text{Energy}_{v,k}(s)$, and $\text{TimeNorm}_{v,k}(s)$ (Equation ??) – are first combined into a single resource score, analogous to $\text{Env}_i(x)$:

$$R_{v,k}(s) = \frac{1}{3} (1 - \text{Cost}_{v,k}(s)) + \frac{1}{3} (1 - \text{Energy}_{v,k}(s)) + \frac{1}{3} \text{TimeNorm}_{v,k}(s) \quad (8)$$

$R_{v,k}(s) \in [0, 1]$, with a higher value indicating a better (less costly, faster) candidate (v, k) – the same orientation as $(1 - \text{Env}_i(x))$ in chain routing’s Equation (2). The equal 1/3 weighting is a fixed, untuned starting point, used here for simplicity.

Chain routing’s Equation (2) combines $\text{Sim}_i(x)$ and $\text{Env}_i(x)$ using a single trade-off parameter $\lambda \in [0, 1]$:

$$\text{CombinedScore}_i(x) = (1 - \lambda) \text{Sim}_i(x) + \lambda (1 - \text{Env}_i(x))$$

For DVR, $A_{v,k}(s)$ takes the role of $\text{Sim}_i(x)$ and $R_{v,k}(s)$ takes the role of $(1 - \text{Env}_i(x))$, giving:

$$\text{CombinedScore}_{v,k}(s) = (1 - \lambda) A_{v,k}(s) + \lambda R_{v,k}(s) \quad (9)$$

Mirroring chain routing’s Equation (3), the router selects the verifier and chunk length that maximise the combined score:

$$(v^*(s), k^*(s)) = \arg \max_{(v,k)} \text{CombinedScore}_{v,k}(s) \quad (10)$$

As in chain routing’s Section 5.2, λ is selected by sweeping over $\lambda \in \{0.0, 0.1, \dots, 1.0\}$, running the full pipeline at each value, and choosing the value that gives the best accuracy-resource trade-off (modelled as a Pareto optimisation problem, following Appendix D.4 of chain routing).